

RESEARCH ARTICLE

Introduction to OmniPES: A Novel Modelica Library for Power Systems Modeling and Analysis

MARCELO AROCA TOMIM^{ID}, RICARDO MOTA HENRIQUES^{ID}, (Senior Member, IEEE),
AND JOÃO A. PASSOS FILHO^{ID}, (Senior Member, IEEE)

Department of Electrical Energy, Federal University of Juiz de Fora (UFJF), Juiz de Fora 36036-900, Brazil

Corresponding author: Marcelo Aroca Tomim (marcelo.tomim@ufjf.br)

ABSTRACT This paper introduces OmniPES, a Modelica library tailored for the swift prototyping of small to medium-scale power systems, facilitating analysis in both steady-state and transient conditions. The primary contribution of this new library is to enable the development and enhancement of models in an effective and efficient manner within a Modelica integrated development environment, without the need for external computational software. Notable features include sigmoid-based switches that enforce reactive power constraints in power sources during steady-state or quasi-steady-state operation. Moreover, the library incorporates a transient stability modeling framework, offering a structure akin to those utilized in production-grade power systems programs. These features are illustrated through a tutorial system operating under a secondary voltage regulation loop. The presented results validate and clearly demonstrate the accuracy and effectiveness of the proposed library.

INDEX TERMS Modelica, power systems, secondary voltage control, steady-state analysis, transient analysis.

ACRONYMS

CVRMSE	Coefficient of the Variation of the Root Mean Square Error.
DER	Distributed Energy Resources.
FACTS	Flexible Alternating Current Transmission Systems.
IEA	International Energy Agency.
MRE	Maximum Relative Error.
MSL	Modelica Standard Library.
OECD	Organization for Economic Co-operation and Development.
STATCOM	Static Synchronous Compensator.
SVR	Secondary Voltage Regulator.

I. INTRODUCTION

Operators and Planners of Modern Electric Power Systems have been facing ever-increasing challenges due to the

The associate editor coordinating the review of this manuscript and approving it for publication was Vitor Monteiro^{ID}.

rapid growth in the utilization of DER (Distributed Energy Resources), often associated with renewable energy sources. According to the IEA (International Energy Agency), electricity generated by renewables in the OECD (Organization for Economic Co-operation and Development) countries [1] increased by 80.8 % from 2010 to 2022 at a rate of approximately (128.0 ± 2.6) TWh/year. In contrast, non-renewables generated electricity experienced a 13.8 % decrease in the same period averaging around (97.7 ± 9.2) TWh/year [2]. On the downside, however, renewable energy resources are naturally dispersed throughout electric power grids and often suffer from intermittency, imposing additional challenges for system operators to predict their energy availability and integrate them seamlessly with dispatchable energy resources, such as non-renewables and/or hydropower with reservoirs.

DER are often connected to electric networks through power converters, granting DERs the capability to contribute to the voltage and frequency regulation of the system. These devices, however, require more advanced modeling and

analysis techniques, which must be incorporated into more traditional analytical frameworks, such as transient stability analysis.

In such circumstances, engineering students and professionals must enhance their understanding of power system dynamics by using tools that enable them to rapidly prototype models and integrate them into larger systems, to supplement more traditional approaches. For example, the National Interconnected System, which serves Brazil, experienced a major blackout in August 2023, resulting in the interruption of approximately 23 GW of load. The main causes were identified as issues related to the performance of voltage control associated with wind generation plants at the origin of the event [3]. This incident highlights the need to develop tools that enable the rapid development and validation of control systems. As an example of such tools, the Modelica language was developed with the intent to standardize the mathematical description of complex dynamic systems, which combines continuous and discrete domains [4].

Several Modelica libraries that provide frameworks for power system analysis are available [5], [6], [7], [8], [9], [10], not to mention the variety of models provided by the MSL [11]. In [5] and [6], the OpenIPSL library is discussed and compared with the industrial-grade PSS/E, developed by Siemens. The OpenIPSL library provides users with a vast collection of power system elements and models based on PSAT [12] and PSS/E programs for phasor-domain simulations typical in power systems stability analysis. As another example of such a library, the PowerGrids library [7] was designed with the intent to ease the exchange of open-source models among power systems actors such as manufacturers, operators, and researchers. Both libraries, however, still require power flow results obtained with third-party programs to initialize transient stability simulations.

On another research front, developments in a Modelica-based library for electromagnetic transient simulations are presented in [9] and [10]. Although seemingly unrelated to the phasor-domain models used in the present work, the authors highlight the potential of the Modelica language as a means for facilitating the exchange of power system models between industry and academia in an open-source format. Additionally, advancements in the field of electrical machines using Modelica are discussed in [13], where innovative thermal power generation systems are modeled in the context of the energy transition. In the area of power system controllers, the work in [14] demonstrates the unique modeling capabilities of Modelica, enabling advanced power system modeling for control applications.

In the present development, one of the first aims was to learn how the Modelica language could be effectively used for power system modeling and analysis. As the library matured, it was structured along the same lines as industrial-grade power-flow and transient stability programs employed for modeling bulk power systems. At the present stage of the development, the OmniPES library provides mainly

two analysis frameworks: steady-state and transient stability. In the first framework, power-flow restrictions are always enforced during a simulation. The second one, on the other hand, considers the premises of transient stability programs, which allow the inclusion of slow dynamics associated with the generation, load, and other system-controlling devices.

Furthermore, within the OmniPES library, initial operating conditions are established through embedded power-flow restrictions applied to power plants and loads. This modeling feature facilitates the rapid prototyping of small to medium-scale power systems, eliminating the necessity of importing power flow results from third-party programs.

Since the library is publicly available as a GitHub repository [15], in this article, the most fundamental features of the OmniPES library are presented using the tutorial system presented in [16] and also used by a CIGRÉ task force in [17]. This particular system was chosen so that the results obtained, both in dynamic and steady-state conditions, can be replicated and readily compared with those already published. Additionally, the SVR (Secondary Voltage Regulator) implemented for this system has allowed us to showcase the level of flexibility achievable with the proposed Modelica library for modeling both conventional and unconventional power systems.

A. MOTIVATION AND CONTRIBUTION OF THIS WORK

The main contributions of this work are twofold. Firstly, the generators' reactive power limits are modelled using sigmoid-based switches, as described in [18], within the Modelica language. This model enables precise restriction of reactive power output by the sources while adjusting the controlled variables as required by the system's solution, including the backoff procedure from reactive power limits. Secondly, initial conditions for transient or quasi-steady-state simulations are implicitly defined by embedded power-flow restrictions, eliminating the need for users to import power flow results from external third-party tools.

In the paper's sequence, steady-state simulations for the tutorial system are first discussed, examining its operation with and without secondary voltage regulation when subjected to a load increase. Subsequently, dynamic simulations for the same tutorial system are presented. In addition to the preceding discussion on simulation preparation, it is also explored the object-oriented strategy employed for modeling power plants, which can serve as a template for modeling other devices in the future.

All analyzed Modelica libraries for power system simulation relied on third-party load-flow programs to establish steady-state operating conditions for initializing transient stability simulations. In contrast, the OmniPES library was designed from the beginning with automatic power system initialization as a core feature, aiming at simplifying the user experience. This enables users to prototype power systems for various studies using a single simulation tool. As a result, the OmniPES library provides two modelling

frameworks: steady-state and transient stability. In the steady-state framework, power flow constraints are continuously enforced, while in the transient stability framework, these constraints are applied only during the computation of initial conditions. Additionally, these frameworks can be seamlessly combined, enabling long-term simulations to be conducted entirely within the same library.

II. PACKAGE DESCRIPTION

In Fig. 1a, an overview of the OmniPES library is shown. Among its subpackages, one can identify the subpackages `Circuit`, `SteadyState`, and `Transient` as the most important ones in the library, while the others contain auxiliary models and tools such as mathematical functions, unit definitions, and scopes.

The `Circuit` subpackage contains the most basic network elements for positive-sequence power system analysis, such as electrical buses, shunt and series components used to construct more complex models such as voltage and current sources, transmission lines, two-winding transformers, breakers, and faults.

In the `SteadyState` subpackage, one can find models that, together with the `Circuit` subpackage, allow one to perform traditional power flow analysis based on positive-sequence equipment models. In this package, the focus remains on loads and sources, typically specified by means of active and reactive power and voltage magnitudes. Models for various types of power sources are available such as $V\theta$, PV, PQ, and PQ with reactive power limits, in addition to non-linear polynomial load, the well-known ZIP loads. For clarity, the models available in the `SteadyState` subpackage allow power flow computations at every time instant. In this context, the `time` can be seen as a parameterization of model variations, which can account for daily load profiles, generation ramps, wind speed profiles, etc.

The `Transient` subpackage provides models for electromechanical stability studies, such as stability models for synchronous machines and their associated controllers, such as voltage and speed regulators, in addition to power system stabilizers and polynomial loads. One important aspect that cannot be disregarded when dealing with electromechanical stability simulations is the establishment of proper steady-state operating conditions. In the OmniPES library, the choice was to embed the power flow restrictions alongside the initial conditions equations, which distinguishes the present development from other power system libraries. In this manner, the user is not required to import initial operating conditions calculated by third-party programs.

Moreover, by appropriately combining models from `SteadyState` and `Transient` subpackages, the OmniPES library also makes it possible to prepare quasi-steady-state simulations to help analyze system behavior in the long-term time range [19]. This kind of simulation usually neglect faster dynamics (e.g., generator damper windings and inner current control loops in power electronic

devices) and retain slower dynamics, such as those related to secondary voltage and/or frequency regulation, boiler control, under-load tap changers, minimum and maximum excitation limiters, load dynamics, among others.

Up to this date, the OmniPES library has been fully developed with the OpenModelica modeling and simulation environment [20]. The latest simulations were all performed using the version 1.24.4. The modelling implementation is based on the recent MSL 4.0.0 [11].

In the next sections, a test system is used to showcase the most fundamental features of the OmniPES library and its potential for the development of more complex systems with power electronics-based equipment.

III. TUTORIAL SYSTEM

As described in [16] and [17], the present tutorial system consists of two power plants that supply a load through a transmission system formed by five branches (three transmission lines and two transformers) and five electrical nodes as shown in Fig. 2. This single-phase diagram was produced in the graphical interface OMEdit, which is part of the OpenModelica simulation platform [20].

The employed SVR scheme consists of a central controller and distributed plant controllers, all based on proportional-integral controllers, which aims at maintaining a specific system bus voltage around a certain magnitude while keeping the reactive power share between the plants proportional to their nominal capacity.

IV. STEADY-STATE ANALYSIS

The steady-state analysis is the most commonly used tool in various studies conducted in electrical power systems. It primarily involves the calculation of nodal voltages, and the power flows in the transmission system, given a specified load level, established active generation dispatch, as well as the network's topology and parameters. In these conditions, both active and reactive powers injected in every node can be calculated by means of the system nodal admittance matrix.

The solution of all complex voltages in a system is usually performed by “power-flow” or “load-flow” programs which model, in a systematic manner, active and reactive powers injected in each electrical node in terms of nodal complex voltages. Most of the current general-purpose power flow analysis programs use different variations of the Newton-Raphson method for its solution [21].

In the traditional power flow formulation [21], each electrical node contributes to the steady-state operating condition definition with two power equations (active and reactive) and four unknowns, i.e., real and imaginary components of both complex injected power and nodal voltage. So, for a system with n electrical buses, $2n$ equations need to be solved for $2n$ unknowns, whereas $2n$ variables need to be specified. Every electrical island, however, requires, at least, an angle reference, because active and reactive powers flowing through electrical branches (e.g., transmission lines and transformers) depend only on angle differences between

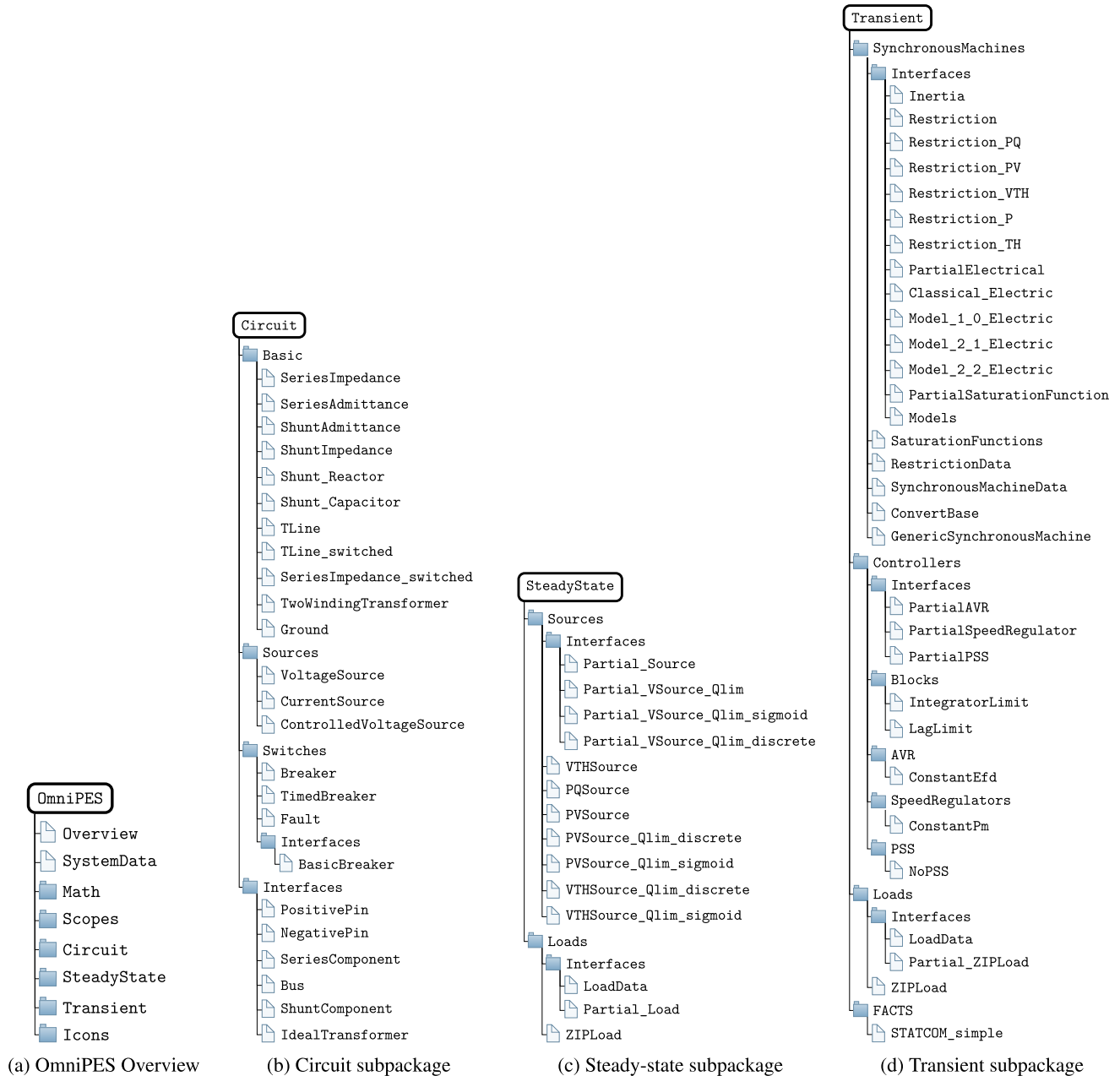


FIGURE 1. OmniPES library structure.

adjacent nodes. Furthermore, the reference bus is also necessary for balancing the power mismatches in the system, including the system losses that are dependent on the currents flowing through the system. This is accomplished by adopting a reference bus for the system, also known as a “swing” or “slack” bus.

So, for each power system, one has to define the slack node, where nodal voltage magnitude and angle are known. For the rest of the electrical nodes, two out of the four variables (voltage magnitude and angle, and injected active and reactive powers) need proper definition. For example, traditional PQ nodes are those where injected active and reactive powers are

known, whereas the complex voltage magnitude and angle is fully unknown.

With this aspect in mind, firstly, `PartialSource` model was implemented as shown in Code 1a, with equations for both active and reactive powers and four unknowns (real and imaginary components of both complex injected power and nodal voltage). Notice that `PartialSource` model itself is inherited from the `ShuntComponent` model that defines the equipment’s drawn current and its voltage drop, detailed in Code 1b. Therefore, in order to complete the `PartialSource` model, two more variables require specification, according to the users’ needs. In the

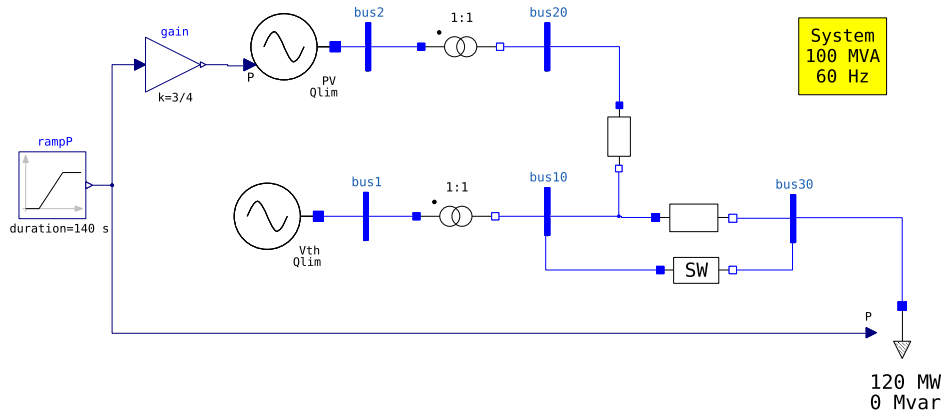


FIGURE 2. Tutorial system implemented for steady-state analysis.

sequence, some of the power-flow-based sources defined in the OmniPES library and their specified variables are presented:

- **VTHSource**: voltage magnitude and angle are defined;
- **PVSource**: active power and voltage magnitude defined, as shown in Code 1c;
- **PQSource**: active and reactive powers defined, according to Code 1d;
- **PVSource_Qlim**: active power and voltage magnitude are defined, while reactive power falls inside its operating range. Otherwise, the reactive power equals the violated limit, while the voltage magnitude becomes an unknown. Further discussion on this source will be set forth in section IV-A.

As for the loads, the previously shown **PQSource**-type and the **ZIPLoad** models can be used. The latter define both active and reactive powers in terms of, at most, second-order polynomials of their terminal voltage magnitude, as shown in equations (1) and (2), where P_0 and Q_0 correspond to the active and reactive power demanded by the load at the voltage magnitude of V_0 . The constants β_p , β_i , and β_z with $\beta \in \{p, q\}$ define the amounts of constant power, constant current, and constant impedance, respectively, composing both the active and reactive power demanded by the load. Additionally, to ensure that the total load remains unchanged, the sum of the constants associated with active or reactive power must equal unity. Finally, the terms ΔP and ΔQ represent variations that may be imposed on the active and reactive power demanded by the load, respectively, during a simulation.

$$P_{load} = (P_0 + \Delta P) \left[p_p + p_i \left(\frac{V}{V_0} \right) + p_z \left(\frac{V}{V_0} \right)^2 \right] \quad (1)$$

$$Q_{load} = (Q_0 + \Delta Q) \left[q_p + q_i \left(\frac{V}{V_0} \right) + q_z \left(\frac{V}{V_0} \right)^2 \right] \quad (2)$$

The above model is implemented as shown in Code 2a, which is inherited from **Partial_ZIPLoad** model given

in Code 2b. The **Partial_ZIPLoad** holds the most basic relationships including the complex power S and the voltage magnitude V . Additionally, the total load can also be modified by the external signals $dpsp$ and $dqsp$, if necessary, as shown in Code 2b.

A. REACTIVE POWER LIMITS

In the traditional formulation of the power flow problem, the equations related to reactive power are not included in the linear system solved at each iteration of Newton's method. This omission can be explained by the fact that the injections of reactive power at PV and V θ buses can be obtained directly through the nodal reactive power equations, assuming that all voltage magnitudes and phases in the system are known.

Reactive power generated by generators or synchronous condensers is calculated after the operational state of the network is computed. Therefore, the traditional method for considering reactive power generation limits involves comparing the generated reactive power to its previously defined limits through the machine's capability curve after achieving convergence or partial convergence in the problem. If the generated reactive power exceeds its limits, the bus is transformed from a PV (voltage-controlled) into a PQ (specified reactive power) bus, where the specified reactive power value is set to the violated limit. Then, the iterative process is resumed.

In the traditional methodology of the power flow problem, this procedure is performed alternately with the iterative process [21]. An alternative way to perform this process is reported in [18], where the equations that model the treatment of limits are incorporated into Newton's method without the need for external modifications in the iterative solution process. This modeling strategy for handling reactive power limits is one that is adopted in the OmniPES library for steady-state analysis and it is shown in Code 3.

In the proposed methodology for representing reactive limits at PV sources and V θ sources, sigmoid-based switches are utilized. These switches transition between the values

(a) Partial_Source abstract model.

```

partial model Partial_Source
  extends Icons.Vsource;
  extends Circuit.Interfaces.ShuntComponent;
  outer SystemData data;
  import Modelica.ComplexMath.conj;
  import Abs=Modelica.ComplexMath.abs;
  import Modelica.Units.SI;
  SI.ComplexPerUnit S "generated apparent power";
  SI.PerUnit V(start=1, max=1.5, min=0.5) "voltage"
  ↪ "magnitude";

  (auxiliary code)

equation
  S = -v*conj(i);
  V = Abs(v);

  (auxiliary code)
end Partial_Source;

```

(b) ShuntComponent abstract model.

```

partial model ShuntComponent
  import Modelica.Units.SI;
  PositivePin p;
  SI.ComplexPerUnit v(re(start=1)) "Node voltage across"
  ↪ "this shunt element.";
  SI.ComplexPerUnit i "Current flowing towards the"
  ↪ "reference node.";

equation
  v = p.v;
  i = p.i;
end ShuntComponent;

```

(c) PVSource model.

```

model PVSource
  extends Interfaces.Partial_Source;
  import Modelica.Units.SI;
  import Abs = Modelica.ComplexMath.abs;
  parameter SI.ActivePower Psp "specified active power";
  parameter SI.PerUnit Vsp = 1.0 "specified voltage"
  ↪ "magnitude";

equation
  S.re = Psp/data.Sbase;
  Vsp = V;
end PVSource;

```

(d) PQSource model.

```

model PQSource
  import Modelica.Units.SI;
  extends Interfaces.Partial_Source;
  parameter SI.ActivePower Psp(displayUnit="MW") "specified"
  ↪ "active power";
  parameter SI.ReactivePower Qsp(displayUnit="Mvar")
  ↪ "specified reactive power";

equation
  S.re = Psp/data.Sbase;
  S.im = Qsp/data.Sbase;
end PQSource;

```

CODE 1. Source model hierarchy.

zero and one according to the evolution of the solution process. In summary, the methodology for handling reactive limits combines all equations of PV sources and V θ sources into a single equation, describing the behavior of generators in both normal operating conditions (while adhering to their reactive power generation limits) and in conditions where these limits are exceeded. The parts of this equation are activated or deactivated through the sigmoid-based switches as the source's operating mode changes.

The equation (3) describes the steady-state behavior of a generator connected to a generic bus k . In this equation, it is possible to observe the behavior of the bus as a PV bus when generation limits are not reached, and the

TABLE 1. PVSource_Qlim operation modes map.

Operation Mode	Sigmoid Switches				Active Equation
	ch1	ch2	ch3	ch4	
Normal	0	0	–	–	$V_k - V_k^{sch}$
Superior Reactive Limit	1	0	1	–	$Q_{G_k} - Q_{G_k}^{max}$
Inferior Reactive Limit	0	1	–	1	$Q_{G_k} - Q_{G_k}^{min}$
Superior Limit Backoff	1	0	0	–	$V_k - V_k^{sch}$
Inferior Limit Backoff	0	1	–	0	$V_k - V_k^{sch}$

behavior as a PQ bus when one of its limits is violated. Note that V_k^{sch} denotes scheduled generator voltage magnitude, V_k is the actual generator voltage magnitude, while $Q_{G_k}^{max}$ and $Q_{G_k}^{min}$ are the maximum and minimum reactive power generation limits, respectively. Switch 1 (ch1) represents the generator identification reaching its maximum reactive power generation limit, while Switch 2 (ch2) identifies its lower limit. The backoff procedure, traditionally used in power flow programs, which is carried out by voltage behavior, is handled through switches 3 and 4 (ch3 and ch4). Switch ch3 identifies whether the bus voltage is above the scheduled value, and ch4 whether the bus voltage is below the scheduled value.

$$\begin{aligned}
 & (1 - \text{ch1} \cdot \text{ch3})(1 - \text{ch2} \cdot \text{ch4})(V_k - V_k^{sch}) + \\
 & + (\text{ch1} \cdot \text{ch3})(1 - \text{ch2} \cdot \text{ch4})(Q_{G_k} - Q_{G_k}^{max}) + \\
 & + (1 - \text{ch1} \cdot \text{ch3})(\text{ch2} \cdot \text{ch4})(Q_{G_k} - Q_{G_k}^{min}) = 0
 \end{aligned} \quad (3)$$

Table 1 shows the mapping of sigmoid-based switches values in both normal operating mode and exceeded reactive limits mode. The implementation of the switches ch1, ch2, ch3, and ch4 are implemented in the algorithm section shown in Code 4.

B. STEADY-STATE RESULTS

To showcase the results obtained with the OmniPES library, the steady-state performance of two voltage control alternatives are compared. The first alternative considers the system operating according to traditional load-flow restrictions where generators control their respective terminal voltage magnitudes. In the second alternative, SVR is employed with bus #30 as the pilot node, while the generators' reactive powers are kept proportional to their respective rated apparent power.

In Fig. 2, generator G1 uses the VTHSource_Qlim model (reference bus with reactive power limits), while G2 uses the PVSource_Qlim model. The load connected to bus #30 utilizes the Ctrl_ZIPLoad model discussed in section IV. Both G2 and the load connected to bus #30 have inputs for an active power ramp that represents a load increase, accompanied by the corresponding generation redispatch. Generator G2 assumes 3/4 of the load ramp, while G1 assumes the remaining 1/4. Reactive power is also divided

(a) ZIPLoad model.

```

model ZIPLoad
  extends Interfaces.Partial_Load;
  parameter Modelica.Units.SI.PerUnit Vdef = 1.0 "Voltage
    ↳ at which the specified power is defined";
  parameter Interfaces.LoadData ss_par =
    ↳ Interfaces.LoadData() "ZIP load parameters";
protected
  parameter Real pp = 1 - ss_par.pi - ss_par.pz;
  parameter Real pi = ss_par.pi;
  parameter Real pz = ss_par.pz;
  parameter Real qq = 1 - ss_par.qi - ss_par.qz;
  parameter Real qi = ss_par.qi;
  parameter Real qz = ss_par.qz;
equation
  S.re = (Psp+dpdp)/data.Sbase*(pp + pi*(V/Vdef) +
    ↳ pz*(V/Vdef)^2);
  S.im = (Qsp+dqsp)/data.Sbase*(qq + qi*(V/Vdef) +
    ↳ qz*(V/Vdef)^2);
end ZIPLoad;

```

(b) Partial_Load abstract model.

```

model Partial_Load
  outer SystemData data;
  extends Circuit.Interfaces.ShuntComponent;
  import Modelica.Units.SI;
  import Modelica.ComplexMath.conj;
  import Abs=Modelica.ComplexMath.abs;
  parameter SI.ActivePower Psp(displayUnit="MW") "Specified
    ↳ active power";
  parameter SI.ReactivePower Qsp(displayUnit="Mvar")
    ↳ "Specified reactive power";
  SI.ComplexPerUnit S "Load complex power";
  SI.PerUnit V(start = 1) "Terminal voltage magnitude";

  Modelica.Blocks.Interfaces.RealInput dPsp if
    ↳ useExternalPsp;
  Modelica.Blocks.Interfaces.RealInput dQsp if
    ↳ useExternalQsp;
  parameter Boolean useExternalPsp = false;
  parameter Boolean useExternalQsp = false;
  Modelica.Blocks.Interfaces.RealOutput dpdp, dqdp;
equation
  S = v*conj(i);
  V = Abs(v);

  if useExternalPsp then
    connect(dPsp, dpdp);
  else
    dpdp = 0;
  end if;

  if useExternalQsp then
    connect(dQsp, dqdp);
  else
    dqdp = 0;
  end if;
end Partial_Load;

```

CODE 2. Load model hierarchy.

in the ratio of 1/4 and 3/4 between generators G1 and G2, respectively, when the SVR is active.

The power flow results for all load ramp points are displayed in Fig. 3, which precisely align with those obtained in [16] and [17]. The dynamic behavior of the generator G1 and the pilot bus is shown in Figures 3a, 3b and 3c. In Fig. 3b, the generator G1, without SVR, reaches its maximum reactive power limit at 26 Mvar when the active load ramp exceeds 200 MW. At this point, a noticeable change occurs in the sigmoid-based switches, as seen in Fig. 4, which forces the generator G1 to transition from voltage control to reactive power control at its maximum limit of 26 Mvar. In particular, ch1 switches from zero to one, and ch4 switches from one to zero, while ch2 and ch3 remain at zero and one, respectively. This transition corresponds to generator G1

```

partial model Partial_VSource_Qlim
  outer SystemData data;
  extends Icons.Vsource;
  extends Interfaces.Partial_Source;
  import Modelica.Units.SI;
  parameter SI.PerUnit Vsp = 1.0 "Specified Terminal
    ↳ Voltage";
  parameter SI.ActivePower Psp(displayUnit="MW") = 0.0
    ↳ "Specified Active Power";
  parameter SI.ReactivePower Qmin(displayUnit="Mvar") =
    ↳ -1e12 "Minimum reactive power";
  parameter SI.ReactivePower Qmax(displayUnit="Mvar") =
    ↳ +1e12 "Maximum reactive power";

  parameter Boolean useExternalPowerSpec = false "Check to
    ↳ activate the external power specification";

  parameter Boolean useExternalVoltageSpec = false "Check
    ↳ to activate the external voltage specification";

  Modelica.Blocks.Interfaces.RealInput dPsp if
    ↳ useExternalPowerSpec;

  Modelica.Blocks.Interfaces.RealInput dVsp if
    ↳ useExternalVoltageSpec;

  Modelica.Blocks.Interfaces.RealOutput dpdp;
  Modelica.Blocks.Interfaces.RealOutput dvdp;

equation
  if useExternalPowerSpec then
    connect(dPsp, dpdp);
  else
    dpdp = 0;
  end if;

  if useExternalVoltageSpec then
    connect(dVsp, dvdp);
  else
    dvdp = 0;
  end if;
end Partial_VSource_Qlim;

```

CODE 3. Partial_VSource_Qlim model equations.

```

partial model Partial_VSource_Qlim_sigmoid
  outer SystemData data;
  extends Icons.Vsource;
  extends Interfaces.Partial_VSource_Qlim;
  import Modelica.Units.SI;
  parameter SI.PerUnit growth_rate = 1e5 "Sigmoid growth
    ↳ rate";
  parameter SI.PerUnit tolq = 1e-3 "Reactive power
    ↳ tolerance";
  parameter SI.PerUnit tolv = 1e-3 "Voltage magnitude
    ↳ tolerance";

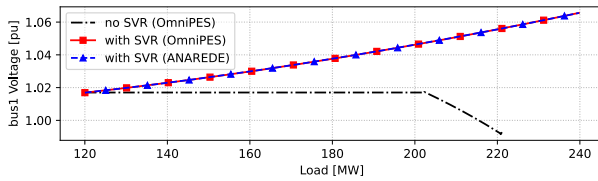
  Real ch1, ch2, ch3, ch4;
  protected
  final parameter Real lim_max = Qmax/data.Sbase - tolq;
  final parameter Real lim_min = Qmin/data.Sbase + tolq;
  Real lim_sup;
  Real lim_inf;
  initial algorithm
    (initialization code)

  equation
    lim_sup = Vsp + dvdp + tolv;
    lim_inf = Vsp + dvdp - tolv;
    (1 - ch1*ch3)*(1 - ch2*ch4)*(V - Vsp - dvdp) + ch1*ch3*(1
    ↳ - ch2*ch4)*(S.im - Qmax/data.Sbase) + (1 -
    ↳ ch1*ch3)*(ch2*ch4)*(S.im - Qmin/data.Sbase) = 0;
  algorithm
    ch1 := 1/(1 + exp(-growth_rate*(S.im - lim_max)));
    ch2 := 1/(1 + exp(growth_rate*(S.im - lim_min)));
    ch3 := 1/(1 + exp(growth_rate*(V - lim_sup)));
    ch4 := 1/(1 + exp(-growth_rate*(V - lim_inf)));
  end Partial_VSource_Qlim_sigmoid;

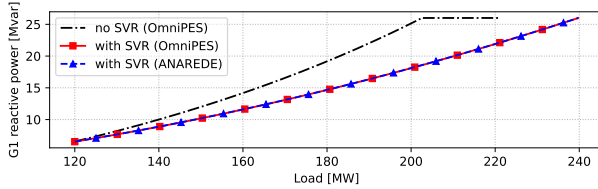
```

CODE 4. Partial_VSource_Qlim_sigmoid model.

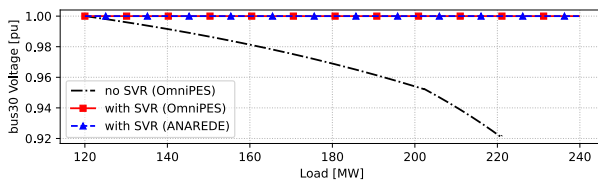
shifting from Normal mode to Superior Reactive Limit mode, as defined in Table 1. The previous operating mode change is



(a) Voltage at G1 terminal.



(b) Reactive power in G1.



(c) Voltage at the load.

FIGURE 3. Steady-state simulations with and without SVR.

also noticeable in 3a, characterized by a voltage drop at the bus #1, the generator G1 terminal.

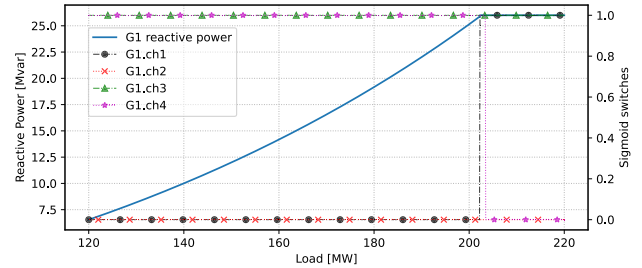
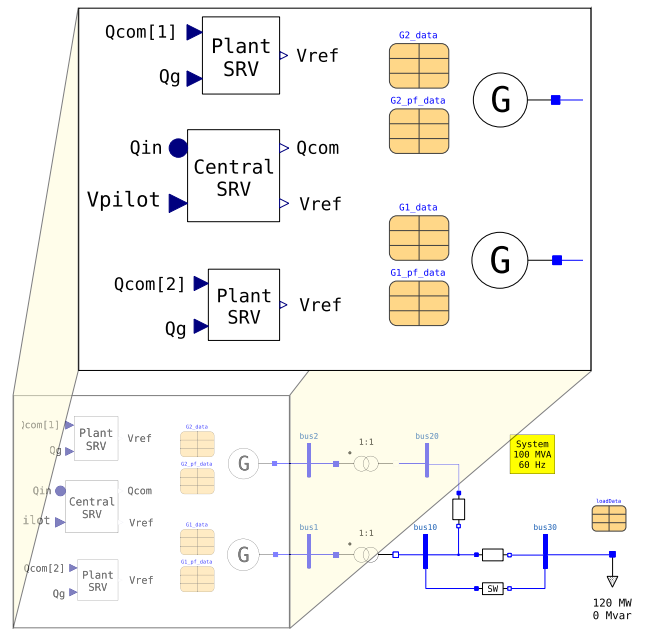
Also, in Fig. 3, results obtained with the OmniPES library on the OpenModelica simulation platform are compared to those obtained with the industrial-grade power flow program ANAREDE, developed by CEPEL (Brazilian Electric Energy Research Center). Although the results are visually very coherent with each other, the CVMSE (Coefficient of the Variation of the Root Mean Square Error) was also calculated. The Coefficient of the Variation of the Root Mean Square Error (CVMSE) is defined in (4) where $\mathbf{y}$ corresponds to a vector with the calculated variable, and $\mathbf{y}^*$ represents another vector with reference values. In the present comparisons, the reference values are those obtained with ANAREDE. Moreover, $E(\mathbf{y}^*)$ returns the average value of the reference variable, which normalizes the root mean square error calculated with n computed and reference values.

$$\text{CVMSE}(\mathbf{y}, \mathbf{y}^*) = \frac{1}{E(\mathbf{y}^*)} \sqrt{\sum_{k=1}^n \frac{(y_k - y_k^*)^2}{n}} \quad (4)$$

The results for the CVMSE calculated for the variables depicted in Fig. 3 are shown in Table 2. One can observe that the computed CVMSE for the voltages and reactive power at the generators remains lower than 0.003 %. Additionally, the MRE (Maximum Relative Error) values, and their corresponding load values, remain lower than 0.005 %.

TABLE 2. Error metrics for the selected steady-state results.

Variable	CVMSE [%]	MRE [%]	Load at MRE [MW]
G1 voltage	0.0026	0.0043	144
G2 voltage	0.0025	0.0044	168
G1 reactive power	0.0002	0.0003	228
G2 reactive power	0.0001	0.0002	228
Load voltage	0.0000	0.0000	120


FIGURE 4. Sigmoid-based switches and reactive power for G1.

FIGURE 5. Modifications for the dynamic analysis.

V. DYNAMIC ANALYSIS

For the dynamic analysis example, the generation part of the steady-state model of the tutorial system shown in Fig. 2 was modified, as shown in system detail depicted in Fig. 5. In this new system, the network and load remain unchanged, while the sources are replaced with synchronous machine-based hydroelectric power plants. In the sequence, the basic strategy for modeling both steady-state restrictions and dynamic behavior of the power plants will be discussed.

(a) Restriction data.

```

record RestrictionData
  extends Modelica.Icons.Record;
  import Modelica.Units.SI;
  parameter SI.ActivePower Psp(displayUnit="MW") = 0.0
    ⇐ "Generated Active Power";
  parameter SI.ReactivePower Qsp(displayUnit="Mvar") = 0.0
    ⇐ "Generated Rective Power";
  parameter SI.PerUnit Vsp = 1.0 "Bus voltage magnitude";
  parameter SI.Angle theta_sp = 0 "Bus voltage angle";
end RestrictionData;

```

(b) Restriction abstract model.

```

partial model Restriction
  outer SystemData data;
  import Modelica.Units.SI;
  parameter RestrictionData param;
  SI.PerUnit P;
  SI.PerUnit Q;
  SI.PerUnit V;
  SI.Angle theta;
end Restriction;

```

(c) Restriction_PV model.

```

model Restriction_PV
  extends Restriction;
  initial equation
    P = param.Psp/data.Sbase;
    V = param.Vsp;
end Restriction_PV;

```

(d) Restriction_PQ model.

```

model Restriction_PQ
  extends Restriction;
  initial equation
    P = param.Psp/data.Sbase;
    Q = param.Qsp/data.Sbase;
end Restriction_PQ;

```

CODE 5. Restriction model hierarchy.

A. EMBEDDED POWER-FLOW RESTRICTIONS

Every power system dynamic simulation requires a set of valid initial conditions, defined in accordance to previously established steady-state restrictions. Therefore, the same concepts discussed in the section IV need to be replicated in the dynamic models, but only for the initialization process of the simulation. In this manner, an abstract *Restriction* model, as shown in Code 5b, serves as the base class for establishing other steady-state restrictions, such as the *Restriction_PV* model, which is detailed in Code 5c.

The abstract *Restriction* model, shown in Code 5b, does not implement any equations; instead, it declares four variables: active power P , reactive power Q , voltage magnitude V and voltage angle θ . The initial equation section is then implemented by derived models, as exemplified in the *Restriction_PV* model (Code 5c). In this model, both active power P and voltage magnitude V are set to their specified values, P_{sp} and V_{sp} , respectively. These specified values are passed to restrictions through a record of type *RestrictionData*, show in Code 5a. The same approach is employed for other restrictions, such as *Restriction_PQ*, shown in Code 5d.

B. POWER PLANT MODELING

For the power plants, another abstract model was created, the *GenericSynchronousMachine* model, illustrated in

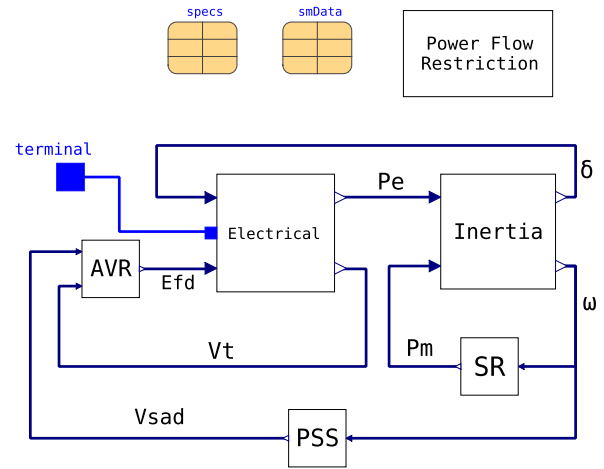


FIGURE 6. GenericSynchronousMachine model.

Fig. 6. This model is composed of other submodels properly interlinked. It's important to note that in this diagram, triangle-shaped arrows represent the unidirectional transmission of causal variables, while square-shaped connections exchange acausal variables, such as voltages and currents.

The electrical model characterizes the dynamics and constraints imposed by the electrical circuits within this machine and produces electrical active power P_e and its terminal voltage magnitude V_t as outputs. The *Inertia* model takes in electrical power P_e , converted by the generator, and mechanical power P_m , developed by the prime mover. It then outputs the angular speed and the load angle with respect to the arbitrary angular reference frame within the network.

The speed regulator (*SR* model) receives the generator's angular speed and delivers the mechanical power P_m developed by the prime mover. Meanwhile, the automatic voltage regulator (*AVR* model) takes in the generator's terminal voltage magnitude V_t and the stabilizing signal V_{sad} , produced by the power system stabilizer (*PSS* model). It's worth noting that, for the present simulations, no *PSS* model was necessary.

Except for the *Inertial* model, all submodels are declared as *partial* abstract models in the Modelica language. This declaration implies that actual implementations are required for each submodel to complete the generator model. As an example, the *PartialElectrical* model is detailed in Code 6a. In this model, only interface restrictions are described, along with stator voltage and flux linkage and the electrical converted power. The number of rotor circuits, however, depends on the generator type, whether it corresponds to a hydro or turbo generator. Specialization is achieved with derived models, such as the *Model_2_1_Electric* model, partially shown in Code 6b, which represents the rotor circuitry formed by a field winding and two damper windings without saturation effects [22]. The naming convention for the electrical models follows the one established in [23].

(a) PartialElectrical model.

```

parameter SI.PerUnit ra = smData.Ra;
parameter SI.PerUnit xl = smData.Xl;
initial equation
der(delta) = 0.0;
equation
Vabs = abs(terminal.v);
theta = arg(terminal.v);
Vqd = sys2qd(terminal.v, delta);
Iqd = sys2qd(-terminal.i, delta);
St = terminal.v*conj(-terminal.i);
Pt = St.re;
Qt = St.im;
Vt = Vabs;
Vqd.re = -ra*Iqd.re + Fqd.im;

```

(b) Model_2_1_Electric model.

```

parameter SI.Time T2d0 = smData.T2d0;
public
initial equation
der(Fld) = 0;
der(Fdi) = 0;
der(F2q) = 0;
equation
if is_saturable then
Fld = sat_d.u;
end if;
T1d0*der(Fld) = Efd - XmdIfd;
T2d0*der(Fdi) = -F2d + Fld - (xld - x2d)*Iqd.im;
T2q0*der(F2q) = -F2q - (xq - x2q)*Iqd.re;
Fdi = F2d - (x2d - xl)/(xld - xl)*Fld;
XmdIfd = -(xd - xld)/(xld - xl)*F2d + (xd - xl)/(xld -
xl)*Fld + (x2d - xl)/(xld - xl)*(xd - xld)*Iqd.im +
↳ (if is_saturable then sat_d.y else 0);

```

CODE 6. Machines electrical model hierarchy.

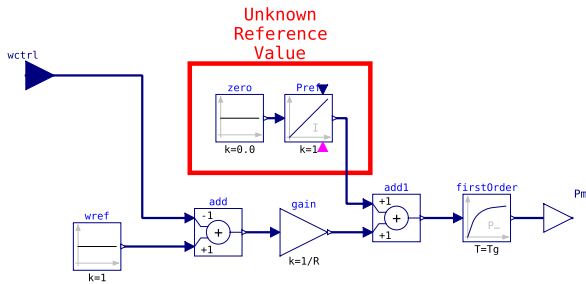


FIGURE 7. SpeedGovernor model.

The actual implementation for the speed regulator is depicted in Fig. 7, for illustration purposes. The speed regulator employs a proportional controller with a droop gain denoted by R , along with a first-order transfer function representing the time-lag imposed by the prime mover.

The voltage regulator is based on the AC4C-type excitation system, as reported in [24]. It is modeled as a single time constant block with non-windup limits [22]. Detailed descriptions of all models can be found in [16] and [17].

It is important to note that the inheritance mechanism, intrinsic to the Modelica language, enables rapid model switching using the `redeclare` statement. This means that any derived model of a parent can be utilized to implement a specific block.

The dynamic models associated with the power plants also rely on reference values for the controllers, which are inevitably linked to the operating conditions defined by the power-flow restrictions. Take, for example, the speed regulator illustrated in Fig. 7, which requires two reference

values: one for angular speed and another for active power. The former was set to unity, while the latter needs to be calculated to ensure that the speed error passed to the first-order block (representing the turbine) is zero in steady-state. In Modelica, an unknown constant variable can be represented by setting its derivative to zero in steady-state, a condition achieved by using an integrator that receives a constant zero input. This way, the state of the integrator associated with the active power reference value can be solved simultaneously with the power-flow restrictions.

C. SECONDARY VOLTAGE REGULATION

The tutorial system, as depicted in Fig. 5, was designed to illustrate the dynamics of an Secondary Voltage Regulator (SVR) scheme, as described in [16] and [17]. In this scheme, a set of voltage control devices, including power plants, static compensators, synchronous condensers, LTC transformers, and others, can be coordinated to effectively regulate voltage at specific buses within an electric network. The primary objective of SVR schemes is to enhance system security by automatically improving voltage profiles during system operation. Typically, the time window for SVR dynamics is on the order of minutes.

The SVR control loop is implemented using external control blocks. The local component of the SVR, shown in Fig. 8a, is realized within the `Plant_SVR` model, utilizing standard Modelica models such as `Add` and `PI` controller blocks. This portion of the SVR control loop regulates the reactive power produced by each generator, resulting in the signal V_{ref} , which is added to the voltage reference of that specific generator. Each generator participating in the SVR scheme is associated with a distinct instance of the `Plant_SVR` model.

The global portion of the SVR, depicted in Fig. 8b, takes inputs from the reactive power generated by every participating generator, represented as the array Q_{in} , and the voltage magnitude of the pilot bus, V_{pilot} , which is controlled. Using these reactive powers, the central SVR controller calculates new commanded reactive powers ($Q_{com}[1]$ and $Q_{com}[2]$) while maintaining the total reactive power produced by the generators.

The interconnection among SVR-related controllers, generators, and the pilot bus is illustrated in Code 7. Each plant's AVR reference is determined by the sum of the reference voltage computed by `central_SVR` and a locally generated voltage reference. The reactive power produced by each plant is transmitted to their respective local SVR controllers, `g1_svr` and `g2_svr`, as well as to the `central_SVR` controller. The reactive power reference for each plant, defined by `central_SVR`, is then conveyed to `g1_svr` and `g2_svr`. Finally, the voltage magnitude of the pilot bus is set equal to that of `bus30`.

Steady-state conditions for the tutorial system with secondary voltage regulation require a few adjustments, easily implemented using the Modelica language and its inheritance mechanism. Firstly, the central SVR imposes that the voltage

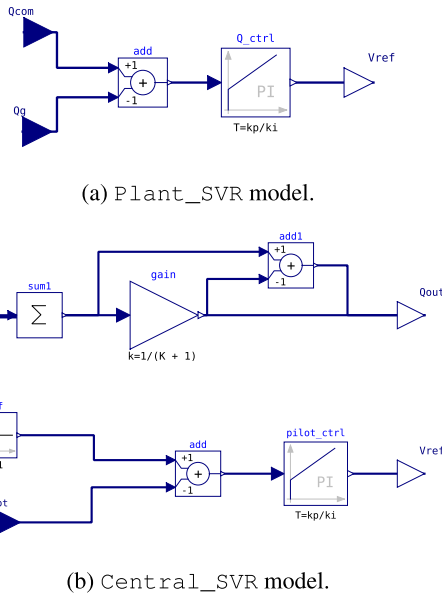


FIGURE 8. SVR controllers.

```

G1.avr.Vext = g1_srv.Vref + central_SVR.Vref;
G2.avr.Vext = g2_srv.Vref + central_SVR.Vref;
g1_srv.Qg = G1.electrical.Qt;
g2_srv.Qg = G2.electrical.Qt;
central_SVR.Qin = {G1.electrical.Qt, G2.electrical.Qt};
central_SVR.Qout = {g1_srv.Qcom, g2_srv.Qcom};
central_SVR.Vpilot = bus30.V;

```

CODE 7. Interconnection among plant and central SVR controllers.

magnitude at the load bus must be fixed. Consequently, at the load bus, the voltage magnitude, active power, and reactive power are known, leaving only the voltage phase as unknown. Thus, with three out of four variables of a single electrical node known, the voltage magnitude at the terminal of one of the power plants is required to remain unknown, dependent on the voltage magnitude at the load.

Additionally, the central SVR also requires that the reactive power output of each power plant must be proportional to its rated apparent power. Consequently, the voltage magnitude in a second power plant is treated as an unknown. In summary, in terms of power-flow nomenclature, the load bus is transformed into a PQV-bus, and the reactive power output of one power plant is made dependent on the other. As a result, the power plants' buses become P and θ buses. This transformation is necessary due to the need for an angular reference in the system.

D. SIMULATION RESULTS

The dynamic behavior of the tutorial system began from a steady-state condition, in which the voltage at bus30 was set to 1 pu. The distribution of reactive power for the power plants was configured so that power plant G1, connected to bus 1, produced one-third of the reactive power generated by power plant G2 [16], [17].

For the sake of accuracy checking, obtained results are also compared to those produced by the industrial-grade transient

TABLE 3. Initial values for selected variables in pu.

Variables	No SVR	With SVR
bus1.V	1.01700	1.01691
bus2.V	1.02500	1.02500
bus10.V	1.00586	1.00582
bus20.V	1.01352	1.01350
bus30.V	1.00004	1.00000
G1.electrical.Pt	0.30000	0.30000
G2.electrical.Pt	0.90000	0.90000
G1.electrical.Qt	0.06548	0.06522
G2.electrical.Qt	0.19537	0.19565
g1_srv.Vref	—	0.00000
g2_srv.Vref	—	0.00811
central_SVR.Vref	—	1.03108

stability program ANATEM [25], [26], [27], [28], [29], also developed by CEPEL.

For the initial condition calculations, power plant G2 defines its dispatched active power, while G1 establishes the angular reference. During the voltage reference calculations, either of the plant SVRs, g1_srv or g2_srv, may explicitly set the derivative of Vref to zero, but not both simultaneously.

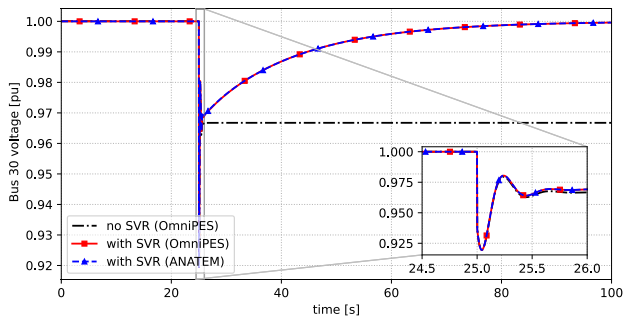
The results for the initial conditions of the tutorial system are presented in Table 3. It is evident that at the beginning of the simulation with the SVR, the pilot bus voltage (bus30) was successfully set to unity while maintaining the one-third ratio for the reactive power generated by G1 and G2. However, in the original operating conditions proposed for the tutorial system in [16], without the SVR, achieving these goals required manual adjustments only at the beginning of the simulation, as indicated by the similarity of the values in both columns in Table 3.

The tested contingency involved opening one of the transmission lines after 25 s of steady-state operation. As depicted in Fig. 9a, the system's SVR successfully regulates the voltage at the pilot bus. Without SVR, the voltage would tend to settle at a significantly lower magnitude. Additionally, the power plants are required to produce less reactive power when compared to the system without SVR. This reduction is mainly due to the decreased reactive power produced by the power plant G1 under SVR control.

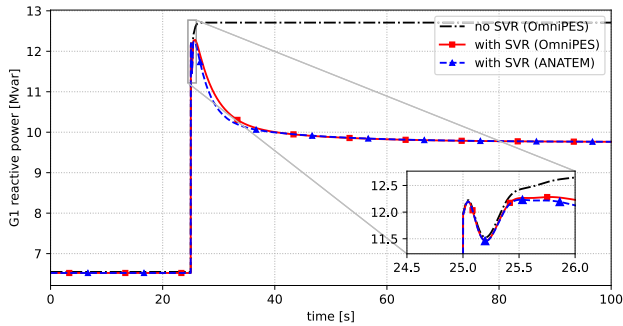
The curves depicted in Fig. 9 exhibit acceptable coherence between OmniPES and ANATEM. This observation is further reinforced by the error metrics (see section IV-B for metrics definitions) reported in Table 4. Notably, the CVRMSE for the load voltage is lower than 0.002 %, while for the reactive powers and load angle, it remains below 0.6 %. Among the variables observed, the Maximum Relative Error (MRE) of 3.29 % is recorded for the relative load angle at 28.49 s.

VI. COMPARISON WITH OPENIPESL

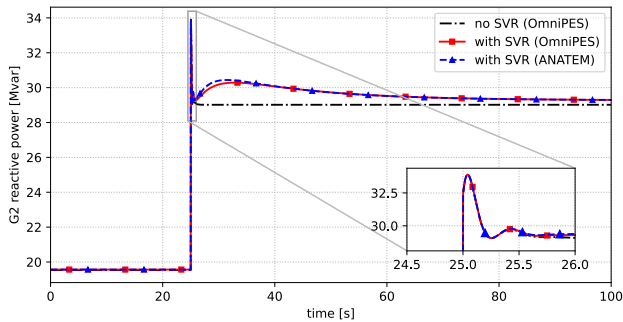
In this section, the OmniPES library is compared with another Modelica library designed for modeling phasor



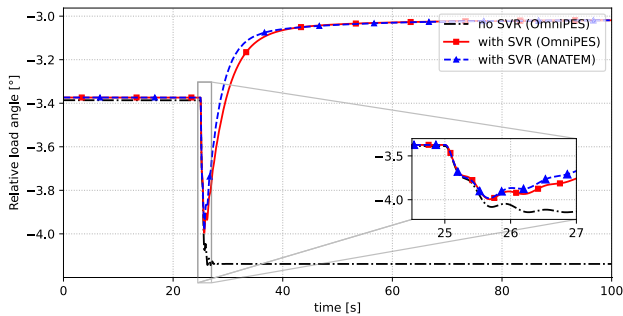
(a) Voltage magnitude at the pilot bus.



(b) G1 reactive power.



(c) G2 reactive power.



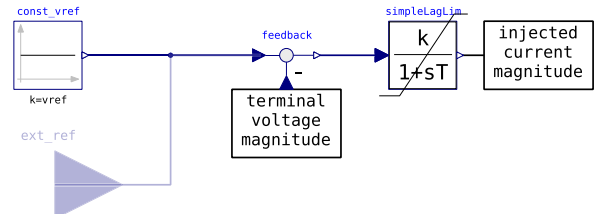
(d) Relative load angle.

FIGURE 9. Selected dynamic simulation results.

time-domain simulations: OpenIPSL [5], [6]. There are two main reasons why OpenIPSL was chosen for this comparison. First, OpenIPSL provides models for a wide range of power system components, including DER and FACTS. Second, this

TABLE 4. Error metrics for the results shown in Fig. 9.

Variable	CVRMSE [%]	MRE [%]	Instant at MRE [s]
Load Voltage	0.0015	0.0108	27.1660
G1 reactive power	0.4227	2.3054	28.3660
G2 reactive power	0.1404	0.8249	28.3700
Load angle	0.5650	3.2896	28.4930


FIGURE 10. Graphical view of STATCOM_simple model.

library has already been validated against production-grade stability analysis tools, such as Siemens PSS/E, as reported in [5] and [6].

The test system used for the comparisons is the IEEE 9-bus system with a Static Synchronous Compensator (STATCOM) device connected to bus 8, as shown in Fig. 11. The simulation was performed for a 100-ms short-circuit at bus 9, followed by the trip of the transmission line interconnecting bus 8 and bus 9. The system was simulated over a 20 s with a 1-ms time step and a tolerance of 10^{-6} . The error metrics used for the comparison are the CVRMSE and the MRE, discussed in section IV-B.

The implemented STATCOM model corresponds to the simple model described in [30]. In this model, the STATCOM is represented by a current source controlled by a proportional controller as depicted in Fig. 10 and detailed in Code 8. This assumption is valid since the current loop response of the real device is usually much faster than the voltage loop response. The STATCOM control system usually aims at regulating the voltage at its terminal to a reference value by means of a pure reactive power injection.

One interesting aspect of the STATCOM model is the use of a limited first-order transfer function to model the proportional controller. Since this specific transfer function is not available in the Modelica Standard Library, it was implemented as part of the OmniPES library. The LagLimit model is shown in Code 9. The model strategy for this limited first-order transfer function is presented in [22, pg. 360] and can be summarized as follows: while the state variable y is within the limits of y_{Min} and y_{Max} , the output y is generated by integrating the input error signal e . When the state variable y reaches either of the limits, the output y is kept constant at the corresponding limit value by stopping the integration. While fixed at the maximum limit, the integration is resumed only when the error signal becomes negative. An analogous procedure is applied when the state variable y reaches the minimum limit, but with the

```

model STATCOM_simple
import Modelica.Units.SI;
import Modelica.ComplexMath.abs;
import Modelica.ComplexMath.conj;
extends OmniPES.Circuit.Interfaces.ShuntComponent;
parameter SI.PerUnit k = 100 "controller gain";
parameter SI.Time T = 0.1 "controller time constant";
parameter SI.PerUnit imax = 1.2 "maximum injected
  ↳ current";
parameter SI.PerUnit imin = -0.8 "minimum injected
  ↳ current";
parameter Boolean external_reference = false;
parameter SI.PerUnit vref = 1.0 "constant voltage
  ↳ reference";
Modelica.Blocks.Math.Feedback feedback;
Modelica.Blocks.Sources.Constant const_vref(k = vref) if
  ↳ not external_reference;
Modelica.Blocks.Interfaces.RealInput ext_ref if
  ↳ external_reference;
OmniPES.Transient.Controllers.Blocks.LagLimit
  ↳ simpleLagLim(k = k, T = T, ymax = imax, ymin = imin);
SI.ComplexPerUnit St;
SI.PerUnit I, V;
equation
I = simpleLagLim.y;
V = abs(p.v);
feedback.u2 = V;
St = -p.v*conj(p.i);
St.re = 0;
St.im = V*I;
connect(feedback.y, simpleLagLim.u);
if external_reference then
  connect(ext_ref, feedback.u1);
else
  connect(const_vref.y, feedback.u1);
end if;
end STATCOM_simple;

```

CODE 8. Modelica code for the STATCOM_simple model.

```

model LagLimit
extends Modelica.Blocks.Interfaces.SISO(y(start=1));
import Modelica.Units.SI;
parameter SI.PerUnit k "constant gain";
parameter SI.Time T "time constant";
parameter SI.PerUnit ymax "maximum limit";
parameter SI.PerUnit ymin "minimum limit";
Boolean is_normal(start=true) "normal operation state";
Boolean hit_max(start=false) "maximum limit operation
  ↳ state";
Boolean hit_min(start=false) "minimum limit operation
  ↳ state";
Real e "signal to be integrated";
initial equation
der(y) = 0;
equation
T*e = k*u - y;
der(y) = if is_normal then e else 0;
algorithm
hit_max := (y > ymax) or (pre(hit_max) and e > 0);
hit_min := (y < ymin) or (pre(hit_min) and e < 0);
is_normal := (y >= ymin and y <= ymax) or (hit_max and e
  ↳ < 0) or (hit_min and e > 0);
end LagLimit;

```

CODE 9. LagLimit model.

integration being resumed when the error signal becomes positive.

A. VALIDATION RESULTS

The comparison results are presented in Table 5 and Fig. 12. The error metrics indicate that the OmniPES library provides results that are very close to those obtained with the OpenIPSL library. The CVRMSE and MRE values for the relative angle δ_{12} , frequency ω_1 , active power P_1 , and voltage at bus 8 are all within acceptable limits, demonstrating the accuracy of the OmniPES library.

TABLE 5. Error metrics in comparison with OpenIPSL.

Variable	CVRMSE [%]	MRE [%]	Instant at MRE [s]
Relative angle δ_{12}	0.0089	0.0245	18.8940
Frequency ω_1	0.0001	0.0003	19.0550
Active power P_1	0.0284	0.0814	18.8890
Voltage at bus 8	0.0000	0.0003	17.5150

Fig. 12 presents the time-domain simulation results for the relative angle between generators 1 and 2, the electric frequency at generator 1, the active power at generator 1, and the voltage at bus 8. From a system performance perspective, the results show that the STATCOM effectively regulated the voltage at bus 8, keeping it close to the reference value, as shown in Fig. 12d. Regarding the comparison between libraries, the plots illustrate that the dynamic responses of the OmniPES and OpenIPSL models are in strong agreement, further validating the accuracy of the OmniPES library.

B. PERFORMANCE COMPARISON

In order to assess the performance of the OmniPES library, a comparison was made between the simulation time for the previously described test system modelled with OmniPES and OpenIPSL libraries. The simulation time was divided into three parts: the event-handling time, the simulation time and other. The event-handling time corresponds to the time spent by the simulation environment to handle the state events, while the simulation time refers to the time spent to actually solve the differential-algebraic equations. The other category includes the time spent on other tasks, such as initialization and output file creation. The simulations were performed on a computer with the following specifications:

- **Processor:** Intel® Core™ i7-9700 CPU, 8 × 3GHz
- **RAM:** 16 GB DDR4
- **Operating System:** Ubuntu 24.04.2 LTS
- **Linux Kernel:** 6.8.0-36-generic
- **C Compiler:** Clang 18.1.3
- **Simulation Software:** OpenModelica 1.24.4

The performance results, presented in Table 6, show that the OmniPES library enabled a simulation roughly twice as fast as the simulation with the OpenIPSL library. Moreover, with the OmniPES library, the event-handling time is also twice as fast as with the OpenIPSL library, despite the eight state events detected during the simulation with the OmniPES library, one more than the number of state events detected with OpenIPSL library. This is an important result, as the event-handling process is a critical task in the performance of the simulation environment.

Another aspect that also influences the performance of the simulation is the number of equations and variables in the model. With the OmniPES library, the test system model has a total of 494 equations and variables, with 257 of these being trivial equations. On the other hand, with the OpenIPSL

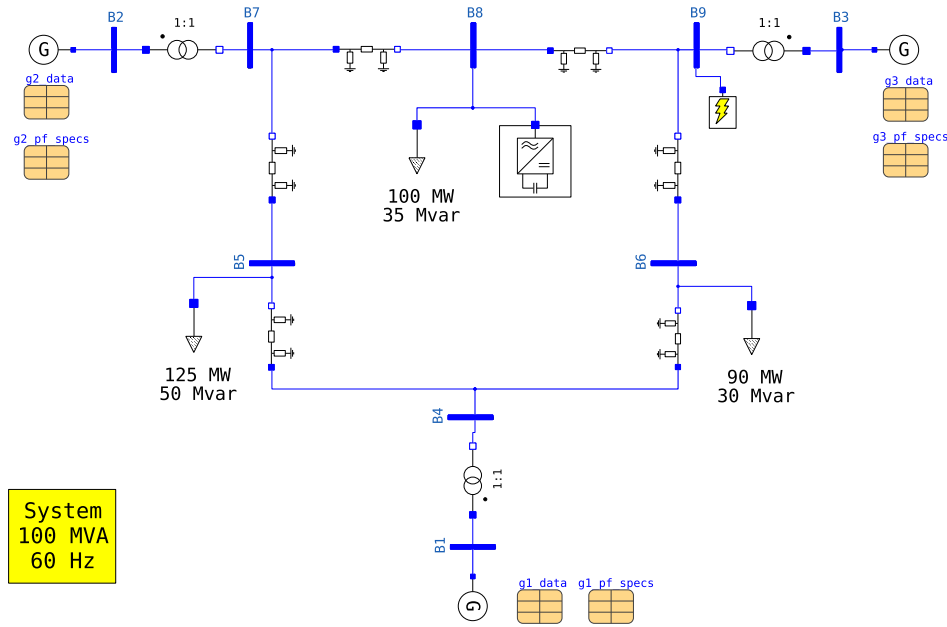


FIGURE 11. IEEE 9-bus system with a STATCOM connected to bus 8.

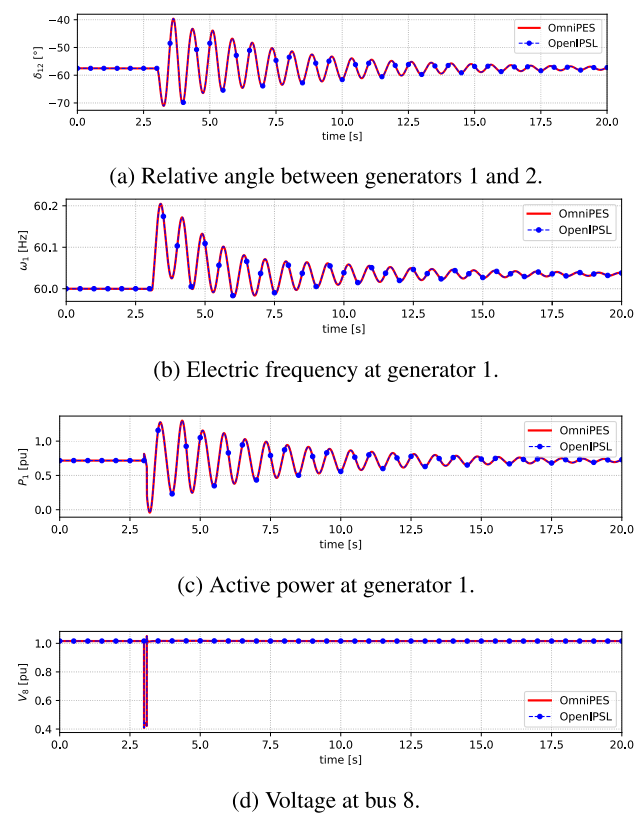


FIGURE 12. Comparison of results between OmniPES and OpenIPSL.

library the test system was modelled with 482 equations and variables, with 240 of these being trivial equations. This difference in the number of equations and variables, however, did not cause a significant impact on the simulation time,

TABLE 6. Simulation timings for OmniPES and OpenIPSL libraries.

Metric	OmniPES [s]	OpenIPSL [s]
Event-handling	1.06	2.10
Simulation	0.81	1.77
Other	0.14	0.14
Total Time	2.01	4.01

as the simulation was performed in roughly half the time with the OmniPES library.

Taking into account that the OmniPES library enabled faster simulations despite having a larger number of equations and variables, it can be concluded that both the OmniPES and OpenIPSL libraries can be further optimized to achieve faster simulations while maintaining their accuracy.

C. OMNIPES USAGE WORKFLOW

It is important to mention that the main goal of the OmniPES library was to provide a user-friendly and self-contained environment for power system simulation. Therefore, one of the main design strategies for the library was taking advantage of the initialization routine embedded in the OpenModelica simulation environment for solving the power flow solution from which a simulation can be started. This approach allows the user to perform a complete simulation of a power system model in a single simulation environment, without the need to switch between different software tools.

Under this light, setting up a simulation with the OmniPES library is straightforward. The user only needs to instantiate the desired power system model along with its constructive parameters and related power flow restrictions. With

OpenIPSL library, however, the user must first perform a power flow analysis using a dedicated software tool, such as GridCal [31], as for the developers recommendation, and generate a Record with the power flow results. This Record is then used as an input for the dynamic simulation.

In this manner, changing the initial operating condition for a simulation requires only the modification of the desired power system model parameters or power flow specifications in the OmniPES library, while with the OpenIPSL library, the user must perform a new power flow analysis and generate a new Record with the updated power flow results.

This difference in the workflow highlights the user-friendly aspect of the OmniPES library, as it simplifies the simulation setup process.

VII. CONCLUSION

In this paper, the Modelica library, OmniPES, is presented. This library is designed for rapid prototyping of small to medium-scale power systems, facilitating both steady-state and transient analysis. The primary objective was to align the library closely with the modeling strategies employed by production-grade power flow and transient stability programs.

Among the notable features of this library, the modeling of generators' reactive power limits using sigmoid-based switches can be highlighted. These switches allow for accurate constraints on reactive power production while adjusting controlled variables as required by the system solution. It is important to note that the sigmoid-based switches also permit the system to backoff from its limits as the system operating conditions allow.

Another noteworthy feature pertains to the initial conditions for transient simulations. These conditions are implicitly defined by embedded power-flow restrictions. Consequently, users are not obligated to import power flow results from external third-party tools.

The implementation in the Modelica language has been thoroughly tested using a small-scale system to evaluate a secondary voltage control scheme. The presented results validate and clearly demonstrate the accuracy and effectiveness of this proposed library for modeling and simulating both traditional and more complex system scenarios.

REFERENCES

- [1] Int. Energy Agency. (Feb. 2024). *Supporting LAC Countries in Their Reform Efforts*. [Online]. Available: <https://www.oecd.org/latin-america/countries/>
- [2] Int. Energy Agency. (Aug. 2023). *Monthly Electricity Statistics*. [Online]. Available: <https://www.iea.org/data-and-statistics/data-product/monthly-electricity-statistics>
- [3] A. F. Bondarenko, M. N. Govorun, and E. I. Satsuk, "About the Brazilian power grid accident on August 15, 2023," *Power Technol. Eng.*, vol. 58, no. 3, pp. 527–534, Sep. 2024, doi: [10.1007/s10749-024-01843-6](https://doi.org/10.1007/s10749-024-01843-6).
- [4] Modelica Assoc. (Mar. 2024). *Modelica Language*. [Online]. Available: <https://www.modelica.org/>
- [5] L. Vanfretti, T. Rabuzin, M. Baudette, and M. Murad, "iTesla power systems library (iPSL): A modelica library for phasor time-domain simulations," *SoftwareX*, vol. 5, pp. 84–88, Jan. 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352711016300097>
- [6] M. de Castro, D. Winkler, G. Laera, L. Vanfretti, S. A. Dorado-Rojas, T. Rabuzin, B. Mukherjee, and M. Navarro, "OpenIPSL 2.0.0—[iTesla power systems library (iPSL): A modelica library for phasor time-domain simulations]," *SoftwareX*, vol. 21, Feb. 2023, Art. no. 101277. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352711022001959>
- [7] A. Bartolini, F. Casella, and A. Guironnet, "Towards pan-European power grid modelling in Modelica: Design principles and a prototype for a reference power system library," in *Proc. 13th Int. Modelica Conf. Regensburg, Germany: Linköping University Electronic Press*, Feb. 2019, pp. 1–15. [Online]. Available: https://ep.liu.se/en/conference-article.aspx?series=ecp&issue=157&Article_No=64
- [8] R. Franke and H. Wiesmann, "Flexible modeling of electrical power systems—The Modelica PowerSystems library," in *Proc. 10th Int. Modelica Conf.*, vol. 96. Lund, Sweden: Linköping University Electronic Press, Mar. 2014, pp. 515–522. [Online]. Available: https://ep.liu.se/en/conference-article.aspx?series=ecp&issue=96&Article_No=54
- [9] A. Masoom, T. Ould-Bachir, J. Mahseredjian, A. Guironnet, and N. Ding, "Simulation of electromagnetic transients with Modelica, accuracy and performance assessment for transmission line models," *Electr. Power Syst. Res.*, vol. 189, Dec. 2020, Art. no. 106799. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0378779620306027>
- [10] A. Masoom, A. Guironnet, A. A. Zeghaida, T. Ould-Bachir, and J. Mahseredjian, "Modelica-based simulation of electromagnetic transients using dynamo: Current status and perspectives," *Electr. Power Syst. Res.*, vol. 197, Aug. 2021, Art. no. 107340. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0378779621003217>
- [11] Modelica Assoc., Github Repository. *Modelica Standard Library (MSL)*. Accessed: Mar. 2025. [Online]. Available: <https://github.com/modelica/ModelicaStandardLibrary>
- [12] F. Milano, L. Vanfretti, and J. C. Morataya, "An open source power system virtual laboratory: The PSAT case and experience," *IEEE Trans. Educ.*, vol. 51, no. 1, pp. 17–23, Jan. 2008.
- [13] M. L. De Pascali and F. Casella, "Modeling innovative thermal power generation systems for the energy transition with Modelica: An open source and flexible solution," in *Proc. IEEE Conf. Control Technol. Appl. (CCTA)*, Aug. 2024, pp. 533–538.
- [14] L. Vanfretti and C. R. Laughman, "Power system modeling for identification and control applications using modelica and OpenIPSL," in *Proc. IEEE Conf. Control Technol. Appl. (CCTA)*, Aug. 2024, pp. 525–532.
- [15] M. A. Tomim, R. M. Henriques, and J. A. P. Filho. (Feb. 2025). *OmniPES: Modelica Library for Power Systems Simulation*. Github repository. [Online]. Available: <https://github.com/marcelotomim/OmniPES>
- [16] N. Martins, J. C. R. Ferraz, S. Gomes, P. E. M. Quintao, and J. A. Passos, "A demonstration example of secondary voltage regulation: Dynamic simulation and continuation power flow results," in *Proc. Conf. Power Eng. Soc. Summer Meeting*, vol. 2, 2001, pp. 791–796. [Online]. Available: <https://ieeexplore.ieee.org/document/970151>
- [17] N. Martins, S. Corsi, C. Taylor, C. Vournas, C. Canizares, G. N. Taranto, G. Fabrice, H. Lefebvre, J. Paserba, J. V. Hecke, J. Sanchez-Gasca, J. C. R. Ferraz, K. Uhlen, L. Pereira, M. Pozzi, M. C. Perdomo, N. Miller, P. Kundur, T. Van Cutsem, and V. Ajjarapu, "Coordinated voltage control in transmission networks," Task Force C4.602, CIGRÉ, Paris, France, Tech. Rep., Feb. 2007. [Online]. Available: <https://www.e-cigre.org/publications/detail/310-coordinated-voltage-control-in-transmission-networks.html>
- [18] J. P. P. Barbosa and J. A. P. Filho, "New methodologies for SVC modeling in the power flow problem based on sigmoid functions," *Electr. Eng.*, vol. 106, no. 1, pp. 971–981, Sep. 2023. [Online]. Available: <https://link.springer.com/article/10.1007/s00202-023-02029-5>
- [19] T. Van Cutsem and C. Vournas, *Voltage Stability of Electric Power Systems*. New York, NY, USA: Springer, 1998.
- [20] Modelica Assoc. (Mar. 2024). *OpenModelica*. [Online]. Available: <https://openmodelica.org/>
- [21] B. Stott, "Review of load-flow calculation methods," *Proc. IEEE*, vol. 62, no. 7, pp. 916–929, 1974. [Online]. Available: <https://ieeexplore.ieee.org/document/1451474>
- [22] P. Kundur, *Power System Stability and Control* (EPRI Power System Engineering Series). New York, NY, USA: McGraw-Hill, 1994.
- [23] *IEEE Guide for Synchronous Generator Modeling Practices and Parameter Verification With Applications in Power System Stability Analyses*, IEEE Standard 1110-2019, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/9020274>

- [24] *IEEE Recommended Practice for Excitation System Models for Power System Stability Studies*, IEEE Standard 421.5-2016, 2016, p. 207. [Online]. Available: <https://ieeexplore.ieee.org/document/7553421>
- [25] R. L. Miranda, L. Do E. S. Fernandes, D. N. Araujo, R. C. Medrado, D. F. G. Filho, and A. G. M. Junior, "Application and validation of WECC models for photovoltaic and energy storage systems in ANATEM software for power system stability studies," in *Proc. IEEE PES Innov. Smart Grid Technol. Latin Amer. (ISGT-LA)*, Nov. 2023, pp. 210–214. [Online]. Available: <https://ieeexplore.ieee.org/document/10328254>
- [26] S. C. Lopes and S. G. Di Santo, "Analysis of the Brazilian operating reserve through instantaneous variation of wind generation," in *Proc. IEEE ANDESCON*, Sep. 2024, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10755803>
- [27] M. P. da Silva, F. L. Lirio, S. Gomes, and T. J. M. A. Parreiras, "Evaluation of STATCOM model for electromagnetic and electromechanical transients," in *Proc. Simposio Brasileiro de Sistemas Eletricos (SBSE)*, May 2018, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/8395743>
- [28] C. C. Soto, A. P. Grilo Pavani, A. G. Moraco, and R. A. Ramos, "Analysis of wind farms frequency support," in *Proc. IEEE PES Innov. Smart Grid Technol. Conf.-Latin Amer. (ISGT Latin Amer.)*, Sep. 2021, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/9543009>
- [29] M. Camponogara and D. P. Bernardon, "Evaluation of automatic generation control modes using ANATEM software," in *Proc. IEEE PES Innov. Smart Grid Technol. Conf.-Latin Amer. (ISGT Latin Amer.)*, Sep. 2019, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/8895280>
- [30] F. Milano, *Power System Modelling and Scripting* (Power Systems). Berlin, Germany: Springer, 2010.
- [31] S. P. Vera. (Feb. 2025). *GridCal*. Github repository. [Online]. Available: <https://github.com/SanPen/GridCal>



MARCELO AROCA TOMIM received the B.Sc. and M.Sc. degrees in electrical engineering, major in power systems from the Federal University of Itajubá (UNIFEI), in 2001 and 2004, respectively, and the Ph.D. degree from The University of British Columbia (UBC), Vancouver, Canada, in 2009. Since 2010, he has been with the Federal University of Juiz de Fora (UFJF), Brazil. His research interests include modeling, simulation, and control of power systems.



2013, he has been an Associate Professor with the Electrical Engineering Department, UFJF. His research interests include voltage security analysis and control of power systems.

RICARDO MOTA HENRIQUES (Senior Member, IEEE) received the B.Sc. degree from the Federal University of Juiz de Fora (UFJF), in 1997, and the M.Sc. and D.Sc. degrees from the COPPE, Federal University of Rio de Janeiro, in 2002 and 2009, respectively. He was with Brazilian Electric Energy Research Center (CEPEL), from 1997 to 2013, mainly on computational methods for bulk power systems analysis, with an emphasis on steady state analysis methods. Since



His research interests include voltage security analysis, optimization, and control of power systems.

JOÃO A. PASSOS FILHO (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees from the Federal University of Juiz de Fora (UFJF), Brazil, in 1995 and 2000, respectively, and the D.Sc. degree from COPPE, Federal University of Rio de Janeiro, in 2005. From 1997 to 2009, he was with the Electric Energy Research Center (CEPEL), on the development of power system analysis software. Since 2009, he has been with the Department of Electrical Engineering, UFJF.

...